

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Índice general

9. Automatización y Gestión de Recursos	1
9.1. Configuración Básica de Hadoop	1
9.2. Automatización de procesos con Oozie	4
9.2.1. Tipos de Trabajos Oozie	4
9.2.2. Ejemplo de programación de trabajos	6
9.2.3. Planificación de trabajos	10
9.2.4. Paquetes de Trabajos	12
9.2.5. Interfaz de Línea de Comandos	13
9.3. Gestión de recursos con YARN	14

Listado de acrónimos

HDFS	Hadoop Distributed File System
JAR	Java ARchive
YARN	Yet Another Resource Negotiator

9

Capítulo

Automatización y Gestión de Recursos

Dentro de este capítulo se comienza con la configuración básica de una distribución Hadoop. Estos valores son claves, quizás no para su configuración en una distribución dada pero si después para su uso para la programación y distribución de trabajos.

Posteriormente se trató la automatización y la planificación de trabajos con la herramienta Oozie [Vad16]. Apache Oozie es una aplicación que permite programar trabajos dentro del ecosistema Hadoop. Oozie combina múltiples trabajos de forma secuencial en una única unidad de trabajo lógica.

Finalmente, se describirá de forma breve YARN[Voh16], el componente de Hadoop responsable de la gestión de los recursos del clúster y de la planificación de los trabajos.

9.1. Configuración Básica de Hadoop

Los siguientes archivos de configuración son básicos para el funcionamiento del cluster, aunque no se entre en la configuración y el montaje del cluster es necesario conocerlos ya que estas propiedades serán importantes tanto para la automatización de trabajos como para la gestión de recursos del cluster^o.

core-site.xml

Este archivo contiene información como el número de puerto que se usa para Hadoop, la memoria asignada para el sistema de archivos, límite de memoria para almacenar los datos, y el tamaño de lectura/escritura. Además define el servidor que contendrá el sistema de ficheros.

```
<configuration>
  <property>
    <name>fs.default.name</name>
    <value>hdfs://localhost:9000</value>
  </property>
  <property>
    <name>hadoop.tmp.dir</name>
    <value>/tmp/hadoop/tmp</value>
  </property>
</configuration>
```

hdfs-site.xml

Este archivo define la configuración del comportamiento del sistema distribuido de ficheros. Para ello contiene información como el factor de replicación de los datos, la ruta tanto de *Namenode* como de los *DataNodes*, etc. las rutas de acceso de los sistemas de archivos locales. En resumen, las ubicaciones en la que se desea contar con el almacenamiento que genere la infraestructura Hadoop.

```
<configuration>
  <property>
    <name>dfs.name.dir</name>
    <value>/tmp/hadoop/name</value>
  </property>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.data.dir</name>
    <value>/tmp/hadoop/data</value>
  </property>
</configuration>
```

En instalaciones standalone (como Google Colab) se configura que la información no esté replicada. En configuraciones en cluster, la información **debe** estar replicada (normalmente con factor 3).

mapred-site.xml

Configura los aspectos básicos de la ejecución de los procesos Map/Reduce.

Será necesario configurar los directorios donde se realiza el MapReduce. También se configurará el gestor de recursos a utilizar, por ejemplo YARN.

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
  <property>
    <name>mapred.job.tracker</name>
    <value>localhost:9001</value>
  </property>
  <property>
    <name>mapred.system.dir</name>
    <value>/hadoop/mapred/system</value>
  </property>
  <property>
    <name>mapred.local.dir</name>
    <value>/tmp/hadoop/tmp_mapred</value>
  </property>
</configuration>
```

yarn-site.xml

Dentro de este archivo se controlan los recursos físicos que se pueden destinar a cada contenedor. Por ejemplo es bueno habilitar la fase de *shuffle* para que se pueda hacer entre las fases de Map y Reduce ya que YARN por defecto no lo incluye.

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.aux-services.mapreduce_shuffle.class</name>
    <value>org.apache.hadoop.mapred.ShuffleHandler</value>
  </property>
</configuration>
```


9.2. Automatización de procesos con Oozie

Cualquier proceso Big Data en Hadoop suele implicar múltiples tecnologías que tienen que implementarse en un determinado orden y manera. A menudo, estas tecnologías también interactúan entre sí. Por ejemplo, un determinado paso n en el flujo de trabajo puede ejecutarse si y sólo si el paso $n - 1$ se ha ejecutado con éxito. Ejecutar manualmente cada uno de estos múltiples pasos requiere mucho tiempo. **Apache Oozie** aborda este problema proporcionando una gestión de dependencias entre diferentes pasos y tecnologías.

Apache Oozie es una aplicación que proporciona un servicio de ejecución de trabajos para el ecosistema Hadoop. Puede ejecutar tanto trabajos basándose tanto en el orden de los mismos como programados en el tiempo. El tipo de trabajos soportados incluye MapReduce, Hive, Sqoop, Spark y Pig. También puede ejecutar trabajos en cascada.

9.2.1. Tipos de Trabajos Oozie

Principalmente, Oozie puede ejecutar dos tipos de trabajos: trabajos de flujo de trabajo (*workflow jobs*) y trabajos de coordinación (*coordination jobs*).

El flujo de trabajo contiene un grupo de trabajos del ecosistema Hadoop que se ejecutan como una sola unidad en un orden especificado. Los trabajos de coordinación proporcionan un servicio de programación para los trabajos del ecosistema Hadoop. Se pueden programar los trabajos Hadoop en función de un tiempo y un evento. Además, se puede lanzar una lista de trabajos de coordinación como un paquete de aplicación Oozie.

Para escribir una aplicación de flujo de trabajo se necesitan los siguientes componentes:

- archivo *job.properties*
- archivo *workflow.xml*
- scripts que ejecutan trabajos dentro del ecosistema Hadoop

Por ejemplo, para ejecutar un trabajo de Pig después de un trabajo de Hive, se necesitará un archivo *job.properties*, un archivo *workflow.xml*, un archivo de script *Pig* y un archivo de script *Hive*.

job.properties

El archivo *job.properties* de una aplicación Oozie contiene principalmente detalles del entorno Hadoop y detalles del flujo de trabajo como el espacio de nombres HDFS, la URL del rastreador de trabajos (*jobtracker*), la ruta de la aplicación de flujo de trabajo, etc.

Algunas propiedades relevantes son las siguientes:

- **fs.defaultFS**: especifica el espacio de nombres HDFS. Esta propiedad está disponible en el archivo `core-site.xml` del que puede extraerse su valor.

```
fs.defaultFS=hdfs://cluster10
```

- **nameNode**: Aunque es una característica en desuso dado que era habitual en versiones anteriores de Oozie, se llegaba a especificar la dirección del NameNode del cluster como sigue:

```
nameNode=hdfs://namenodehostname:8020
```

- **mapreduce.jobtracker.address**: dirección del *jobtracker* del cluster Hadoop. Esta propiedad está disponible en el archivo `mapred-site.xml`.

```
mapreduce.jobtracker.address=cluster10-1:8021
```

- **oozie.wf.application.path**: especifica la ruta de la aplicación del flujo de trabajo en el clúster HDFS.

```
oozie.wf.application.path=${nameNode}/workflows/oozie-examples
```

- **oozie.libpath**: especificar la ruta de las librerías o ficheros jar que se van a utilizar en la aplicación de forma adicional.

```
oozie.use.system.libpath=true  
oozie.libpath=<hdfs_path>
```



El **JobTracker** es el servicio dentro de Hadoop que distribuye las tareas de MapReduce a nodos específicos en el cluster, idealmente los nodos que tienen los datos, o al menos están en el mismo rack.



Añadir entradas a la propiedad **oozie.libpath** significa efectivamente que Oozie configurará esas librerías para que se localicen en `mapred.cache.files` cuando se ejecuten las acciones en su flujo de trabajo. Hadoop se encargará entonces de copiar esos jars a cada nodo del cluster una vez por trabajo, y las tareas se configuran con el jar en la propiedad de configuración `mapred.job.classpath.files`

El archivo **job.properties** se almacena en el sistema de archivos local y puede ser fácilmente adaptado para ejecutar el script Oozie en diferentes clusters.

workflow.xml

El flujo de trabajo se define mediante **workflow.xml**. Contiene dos tipos de etiquetas XML también llamadas nodos: nodos de control y nodos de acción.

Los *nodos de control* especifican la prioridad del trabajo, es decir, qué trabajo debe ejecutarse primero y cuál debe ser el último. Algunos nodos de control son los siguientes

- *Nodo de inicio* (Start Node): Decide qué trabajo de Hadoop debe ejecutarse primero.
- *Nodo final* (End Node): Decide qué trabajo Hadoop debe ejecutarse en último lugar.
- *Nodo Kill* (Kill Node): Aborta un trabajo Hadoop cuando falla.

El nodo de acción decide el trabajo a ejecutar. Soporta acciones de tipo MapReduce, Hive, Pig, Sqoop, Ssh, Distcp, Spark, etc.. Cada trabajo falla o tiene éxito; por lo tanto, cada acción tendrá dos etiquetas más llamadas *OK* y *Error* para decidir la siguiente acción. La etiqueta *OK* decide la siguiente tarea que se ejecutará si la tarea actual tiene éxito, y la etiqueta *Error* decidirá la siguiente acción si la tarea actual falla.

La figura 9.1 muestra cómo funcionan los diferentes nodos. La acción se ejecuta hasta que falla o hasta que llegue al nodo final.

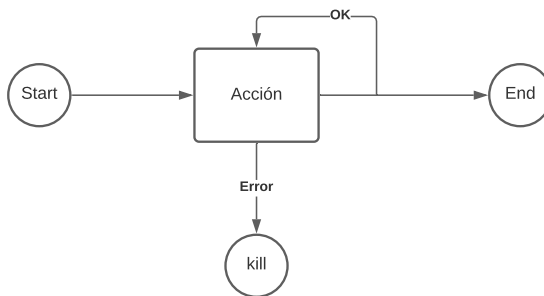


Figura 9.1: Ejemplo de nodos básicos en Oozie

9.2.2. Ejemplo de programación de trabajos

job.properties

La primera tarea para trabajar con Oozie es escribir el archivo **job.properties**. En él se debe especificar la dirección del sistema de archivos y la ruta de la aplicación de flujo de trabajo.

A su vez, también se pueden declarar variables con algún valor para que sean reutilizables dentro de la aplicación Oozie. En el siguiente ejemplo se define una variable llamada `examplesRoot` con ejemplos de valores. También utiliza diferentes variables de entorno para facilitar su configuración como `user.name` que toma el valor del usuario actual del sistema operativo. Si el usuario actual es `cloudera`, entonces `oozie.wf.application.path` se resolverá a `/user/cloudera/examples/apps/pig`. Directorio que debe existir previamente en HDFS.

```
nameNode=\${hadoop.name.node}
jobTracker=\${hadoop.job.tracker}
fs.defaultFS=hdfs://cluster10
mapreduce.jobtracker.address=cluster10n1:8021
queueName=default
examplesRoot=examples
oozie.use.system.libpath=true
oozie.libpath=/user/\${user.name}/\${examplesRoot}/apps/examples-lib
oozie.wf.application.path=/user/\${user.name}/\${examplesRoot}/apps/pig
```

workflow.xml

El archivo `workflow.xml` comienza con la etiqueta XML `<workflow-app>`. El atributo `name` servirá para asignarle un nombre al flujo de trabajo Oozie.

La etiqueta `<start>` especifica el nombre de la acción a ejecutar primero dentro del flujo de trabajo. Esto se realiza en el atributo `to`.

La etiqueta `<start>` va seguida de una lista de etiquetas `<action>` que representan nodos de acción. Cada etiqueta `<action>` tiene etiquetas `<ok>` y `<error>` para representar el éxito y el fracaso. Las etiquetas `<ok>` y `<error>` tienen dos atributos para representar las siguientes etiquetas a ejecutar. La etiqueta `<end>` representa el final del flujo de trabajo. La etiqueta `<kill>` representa el error que se lanzará en caso de que una acción falle.

El listado 9.1 contiene un ejemplo de especificación de flujo de trabajo basado en un trabajo definido en Pig. Esta acción está especificada en el nodo llamado `first-pig-node` y se ejecutará en primer lugar. La acción Pig debe tener la etiqueta `<pig>` dentro de ella. La etiqueta `<script>` contiene la ruta del archivo de script de Pig que debe ser ejecutado. Este archivo de script debe colocarse en el directorio HDFS especificado en el archivo `job.properties` como el valor de `oozie.wf.application.path`. Los parámetros `\${jobTracker}` y `\${nameNode}` se resolverán utilizando sus valores definidos en el archivo `job.properties`.

Listado 9.1: Flujo de Trabajo Oozie para ejecución de un trabajo Pig

```

1 <workflow-app xmlns="uri:oozie:workflow:0.2" name="pig-wf">
2   <start to="first-pig-node"/>
3   <action name="first-pig-node">
4     <pig>
5       <job-tracker>${jobTracker}</job-tracker>
6       <name-node>${nameNode}</name-node>
7       <script>id.pig</script>
8     </pig>
9     <ok to="end"/>
10    <error to="fail"/>
11  </action>
12  <kill name="fail">
13    <message>Pig failed, error message[${wf:errorMessage(wf:lastErrorNo
14    de())}]</message>
15  </kill>
16  <end name="end"/>
17 </workflow-app>

```

Dentro de la configuración de flujo de trabajo se puede utilizar la etiqueta `<configuration>` para especificar un valor a una propiedad. La etiqueta `<configuration>` tiene una etiqueta de propiedad, y ésta a su vez tiene dos etiquetas, `<name>` y `<value>`. En el siguiente ejemplo se muestran diferentes propiedades para configurar un trabajo MapReduce.

```

<configuration>
  <property>
    <name>mapred.job.queue.name</name>
    <value>default</value>
  </property>
  <property>
    <name>mapred.input.dir</name>
    <value>${nameNode}/workflows/oozie-examples/data/*.txt </value>
  </property>
  <property>
    <name>mapred.output.dir</name>
    <value>${nameNode}/workflows/oozie-examples/data/output/wc0 </value>
  </property>
</configuration>

```

Dentro del archivo de flujo de trabajo se pueden especificar diferentes parámetros que se tendrán en cuenta en la ejecución como sucede con la opción `-param` en Pig. Para ello se puede utilizar la etiqueta `<param>` inmediatamente después de la etiqueta `<script>` para pasar un valor a un parámetro, como por ejemplo:

```
<param>INPATH=/user/data/input </param>
```

Además también se pueden especificar los diferentes pasos necesarios para la inicialización de la ejecución de los trabajos del flujo utilizando la etiqueta `<prepare>` que debe ejecutarse antes de lanzar el mismo, por ejemplo:

```
<prepare>
  <delete path="/usuario/cloudera/pig/out"/>.
</prepare>
```

Una aplicación de flujo de trabajo consiste en la definición del flujo de trabajo y todos los recursos asociados como archivos Jar de MapReduce, scripts de Pig, etc.

Las aplicaciones deben seguir una estructura de directorios sencilla y para subirse a HDFS y que Oozie pueda acceder a ellas. Un ejemplo de esta estructura puede ser como sigue:

```
<name of workflow>/</name>
??? lib/
? ??? hadoop-examples.jar
??? workflow.xml
```

A su vez es necesario subir los archivos JAR al directorio HDFS especificado en la propiedad `lib.path` del archivo `job.properties`.

El flujo de trabajo que se ajusta a este diseño puede ser construida con cualquier herramienta de construcción, por ejemplo, Ant o Maven.

Una vez que se dispone de los archivos `job.properties` y `workflow.xml`, éste último y los archivos de scripts (PigLatin, etc.) deben ser subidos al directorio HDFS especificado en `oozie.wf.application.path`. Dado que está organizado en directorios se podría hacer mediante el siguiente comando:

```
hdfs dfs -put hadoop-examples/target/<name of workflow dir> name of workflow
```

De forma individual se podría hacer de la siguiente manera:

```
hdfs dfs -put /path/to/workflow.xml /user/hdfs/examples/apps/pig
hdfs dfs -put /path/to/script.pig /user/hdfs/examples/apps/pig
```

Hay que tener en cuenta que los ficheros relacionados con librerías, scripts, etc. deben ser accesibles a Oozie a partir de la propiedad `lib.path`.

Ejecución de trabajos

La ejecución del flujo de trabajo se puede realizar mediante la interfaz de línea de comandos de Oozie (comando `oozie`).

Algunas de las opciones de ejecución más importantes son las siguientes:

- `-oozie`: especifica la URL del servidor Oozie que se va a utilizar. Se puede omitir siempre y cuando esté configurada la variable de entorno `OOZIE_URL`.
- `-config`: Especifica la ruta del archivo `job.properties` que va a utilizar para la ejecución.
- `-run`: Ejecuta directamente un trabajo.
- `-submit`: Envía el flujo de trabajo al servidor de Oozie, y pone el flujo de trabajo en estado `PREP`.

Ejemplos de ejecución son los siguientes:

```
oozie job -oozie http://localhost:11000/oozie -config job.properties -submit
```

```
export OOZIE_URL="http://localhost:11000/oozie"  
oozie job -config config_workflow.properties -run
```

Si el flujo de trabajo se envía con éxito, devuelve el ID del trabajo Oozie, de lo contrario, devuelve un error.

Una vez iniciado el flujo de trabajo se puede monitorizar desde la consola web o la interfaz de línea de comandos



En las últimas versiones de Oozie los flujos más básicos no requieren la especificación de un `workflow.xml` sino simplemente el propio script de ejecución genera la especificación del flujo de trabajo y se reduce la definición de un requiere que se escriba `workflow.xml` para flujos de trabajo complejos.

9.2.3. Planificación de trabajos

Una vez implementado un flujo de trabajo es siempre importante decidir con qué frecuencia es necesario ejecutarlo: diario, semanal, mensual, trimestral o anualmente.

Para automatizar este trabajo Oozie posee un componente de coordinación que permite ejecutar el flujo de trabajo no sólo en una fecha y hora determinadas, sino también en función de la disponibilidad de los datos.

Como en el caso de la implementación básica se debe especificar un archivo `job.properties`, similar a los anteriores, pero con la inclusión de la propiedad `oozie.coord.application` en lugar de la propiedad `oozie.wf.application.path`. Esta nueva propiedad especificará el directorio HDFS que almacena los archivos del coordinador. El archivo `job.properties` debe estar en el sistema de archivos local como el archivo `workflow`.

Además, se debe implementar un archivo `coordinator.xml` que permite especificar las labores y configuraciones del componente coordinador. Este archivo comienza con la etiqueta `<coordinator-app>` y tiene los siguientes atributos:

- `start`: contiene la fecha y la hora en la que se desea ejecutar la aplicación de flujo de trabajo. Se puede especificar el valor en formato `aaaa-mm-ddThh:mmZ` o se puede un parámetro como `${start}` para poder especificar su valor en el archivo `job.properties`.
- `end`: contiene la fecha y la hora en la que se desea parar la ejecución de la aplicación de flujo de trabajo. Se puede especificar el valor en formato `aaaa-mm-ddThh:mmZ` o se puede un parámetro como `${start}` para poder especificar su valor en el archivo `job.properties`.
- `frequency`: determina la frecuencia con la que desea ejecutar la aplicación de flujo de trabajo. Puede ser mensual, trimestral, anual, etc.
- `timezone`: Especifica la zona horaria en la que desea ejecutar su flujo de trabajo.

Un ejemplo de este fichero de coordinación es el siguiente:

```
<coordinator-app
  name="cron-coord" frequency="\${coord:minutes(10)\}"
  start="\${start\}"
  end="\${end\}"
  timezone="UTC"
  xmlns="uri:oozie:coordinator:0.2"
/>
```



`coord:minutes`, `coord:hours`, `coord:days`, etc. son variables integradas (*built-in*) en Oozie.

Se puede utilizar la etiqueta `<workflow>` dentro de `<coordinator-app>` para especificar el flujo de trabajo que se integrará con el coordinador. La etiqueta `<workflow>` contiene una etiqueta `<app-path>` que contiene la ruta de la aplicación del flujo de trabajo. Por ejemplo, el siguiente código contiene una ruta de la aplicación de flujo de trabajo. Este directorio `hdfs` debe contener el archivo `workflow.xml`; los archivos de código fuente necesarios como Hive, Pig o MapReduce; y los archivos JAR necesarios.


```
<workflow>
  <app-path>/user/cloudera/examples/apps/pig</app-path>
</workflow>
```

Con el fin de registrar o enviar la solicitud de ejecución de trabajo periódico especificada dentro del `coordinator.xml` se puede utilizar la línea de comando con el mismo comando que para el envío de un flujo de trabajo individual. En este caso solo se utiliza el archivo `job.properties` del coordinador.

```
oozie job -oozie http://localhost:11000/oozie -config /path/to/job.
properties -submit
```

Si el envío resulta correcto, se obtendrá el *ID* del trabajo devuelto. Para el flujo de trabajo, el *ID* del trabajo termina con *w*, y para un trabajo de coordinación termina con *c*. Si el envío falla se informa del error correspondiente.

9.2.4. Paquetes de Trabajos

Los paquetes de trabajos de Oozie (*Bundles*) se utilizan para gestionar y lanzar una lista de trabajos de coordinación de Oozie como una única aplicación.

Estos paquetes requieren construir los archivos `job.properties`, ubicado en el sistema de archivos local y `bundle.xml`, ubicado en HDFS.

El archivo `bundle.xml` comienza con la etiqueta `<bundle-app>` y contiene una lista de etiquetas `<coordinator>`. Cada una de estas etiquetas "hijas" deben tener una etiqueta hija llamada `<app-path>` que contiene la ruta del directorio HDFS para la aplicación del coordinador.

Se puede enviar la aplicación Bundle de la misma manera que las aplicaciones de flujo de trabajo y de coordinación utilizando la interfaz de línea de comandos. El archivo `job.properties` debe tener la propiedad `oozie.bundle.application.path`.

```
<bundle-app name='BundleApp' xmlns='uri:oozie:bundle:0.2'>
  <controls>
    <kick-off-time>\${startTime}</kick-off-time>
  </controls>
  <coordinator name='CoordApp-LogParser' >
    <app-path>\${coordAppPathLogParser}</app-path>
  </coordinator>
  <coordinator name='CoordApp-DataExporter' >
    <app-path>\${coordAppPathDataExporter}</app-path>
  </coordinator>
</bundle-app>
```

9.2.5. Interfaz de Línea de Comandos

Oozie proporciona una interfaz de línea de comandos que incluye comandos para la gestión de trabajos, operaciones de administración y comandos específicos para cada tecnología como Hive y Pig, etc. Esta interfaz utiliza internamente la API REST para interactuar con el servidor Oozie.

Oozie proporciona comandos para:

- enviar/submit: enviar un trabajo de Oozie y ponerlo en estado PREP.
- ejecutar/run: ejecución directa de un trabajo de Oozie (estado = RUNNING).
- suspender/suspend: suspender la ejecución del trabajo (estado = SUSPEND).

, ejecutar y suspender trabajos.

El siguiente comando suspende un flujo de trabajo existente:

```
oozie job
  -oozie http://cluster1:11000/oozie -suspend 0000028-1204424112412424-oozie-oozi-W
```

La opción `-kill` permite abortar un trabajo de Oozie especificando el *ID* del trabajo.

```
oozie job
  -oozie http://cluster10-1:11000/oozie
  -kill 0000028-1204424112412424-oozie-oozi-W
```

Se pueden consultar los *logs* del servidor correspondientes a un trabajo específico de Oozie con su *ID* usando la opción de `-logs`.

```
oozie job
  -oozie http://cluster10-1:11000/oozie
  -logs 0000028-1204424112412424-oozie-oozi-W
```

La opción `-info` sirve para obtener más información sobre un trabajo, como la ruta de la aplicación del flujo de trabajo, el estado del trabajo y el *ID* del trabajo externo, incluso el nombre de usuario enviado, la hora de creación, la hora de inicio, la hora de finalización, etc.

9.3. Gestión de recursos con YARN

YARN es una pieza fundamental en el ecosistema Hadoop. Es el framework que permite a Hadoop soportar varios motores de ejecución incluyendo MapReduce, y proporciona un planificador de los trabajos que se encuentran en ejecución en el clúster. Esta mejora de Hadoop se incluyó dentro de la distribución Hadoop 2 [Des16].

Yarn separa las dos funcionalidades principales: la gestión de recursos y la planificación y monitorización de trabajos. Con esta idea, es posible tener un gestor global (Resource Manager) y un Application Master por cada aplicación.

La necesidad de YARN se basa en solventar las desventajas que tenía MapReduce en la arquitectura Hadoop v1. Como ya se ha mencionado, MapReduce, desde su inicio, es un potente marco distribuido y modelo de programación que permitía realizar trabajos paralelos basados en lotes en un clúster de múltiples nodos. A pesar de su eficiencia, en su configuración inicial estaba basado en lotes y, como resultado, no era adecuado para el procesamiento de datos en tiempo real o incluso en tiempo casi real.

YARN dentro de Hadoop v2, se encarga de ejecutar cargo de las partes de gestión de recursos y planificación que en Hadoop v1 se ejecutaban en MapReduce. Principalmente, se encarga de responder a solicitudes de cliente para crear un contenedor y supervisar los contenedores que se están ejecutando y finalizarlos si es necesario para liberar recursos.

Se divide en tres componentes principales que son los siguientes

- **Resource Manager:** El Resource Manager es el componente que orquesta y arbitra sobre el uso de los recursos entre todas las aplicaciones del sistema. El *Resource Manager* mantiene un listado de los *Node Manager* activos y de sus recursos disponibles. Para cumplir esta labor utiliza dos sub-componentes:
 - **Scheduler** o planificador, es el encargado de gestionar la distribución de los recursos del clúster. Además, las aplicaciones usan los recursos que el *Resource Manager* les ha proporcionado en función de sus criterios de planificación. Este planificador no monitoriza el estado de ninguna aplicación ni les ofrece garantías de ejecución.
 - **Application Manager:** es el componente responsable de aceptar las peticiones de trabajos, localizar el primer contenedor en el que ejecutar la aplicación y proporcionar reinicios de los trabajos en caso de que fuera necesario debido a errores.
- **Node Manager:** gestiona los trabajos con las instrucciones del *Resource Manager* y proporciona los recursos computacionales necesarios para las aplicaciones en forma de contenedores. Los contenedores Yarn tienen una asignación de recursos (cpu, memoria, disco y red) fija de un host del clúster y el Node Manager es el encargado de monitorizar esta asignación. Además se encarga de mapear las variables de entorno necesarias, las dependencias y los servicios necesarios para crear los procesos de ejecución.

- **Application Master:** Es el responsable de negociar los recursos apropiados con el *Resource Manager* y monitorizar su estado y su progreso. También coordina la ejecución de todas las tareas en las que puede dividirse su aplicación [Hol14].

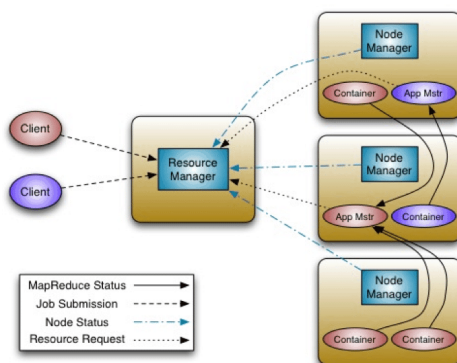


Figura 9.2: Arquitectura YARN. Fuente: Apache

Para ejecutar una aplicación en YARN, el cliente contacta con el Resource Manager y le pide que ejecute un proceso Application Master. El Resource Manager encuentra un Node Manager que puede iniciar la Application Master en un contenedor. Precisamente lo que hace el Application Master una vez que se ejecuta depende de la aplicación, es decir, simplemente puede, o bien, ejecutar una operación en el contenedor en el que se está ejecutando y devolver el resultado al cliente, o bien solicitar más contenedores al Resource Manager y usarlos para ejecutar una operación distribuida.

YARN admite la reserva de recursos a través del **ReservationSystem**, un componente que permite a los usuarios especificar un perfil de recursos en el tiempo y con restricciones temporales (por ejemplo, plazos), y reservar recursos para garantizar la ejecución predecible de trabajos importantes.

Para escalar YARN más allá de un número pequeño de miles de nodos, soporta la noción de Federación a través de la función YARN Federation. La federación permite conectar de forma transparente múltiples (sub-)clusters hacerlos aparecer como un único cluster masivo. Esto se puede utilizar para lograr una mayor escala, y/o para permitir que varios clusters independientes se utilicen juntos para trabajos muy grandes.

Bibliografía

- [Des16] Tanmay Deshpande. *Hadoop Real-World Solutions Cookbook*. Packt Publishing Ltd, 2016.
- [Hol14] Alex Holmes. *Hadoop in practice*. Simon and Schuster, 2014.
- [Vad16] Balaswamy Vaddeman. *Beginning Apache Pig*. Springer, 2016.
- [Voh16] Deepak Vohra. *Practical Hadoop ecosystem: a definitive guide to hadoop-related frameworks and tools*. Apress, 2016.